Advanced Day 1 — Session 1 (Hours 1–4)

Python Programming (Advanced) • Kickoff, Class Design, Validation, and Composition

Session 1 Overview

Topics Covered Today

- Hour 1 — Advanced kickoff, baseline diagnostic, and Git workflow
- Hour 2 — Designing classes from requirements and boundaries
- Hour 3 — Properties, invariants, and custom exceptions
- Hour 4 — Composition vs inheritance and polymorphism

Source Alignment

- `Advanced/lessons/lecture/Day1_Hour1_Advanced.md` → `# Day 1, Hour 1: Advanced Kickoff, Baseline Diagnostic, and Git Workflow`
- `Advanced/lessons/lecture/Day1_Hour2_Advanced.md` → `# Day 1, Hour 2: Designing Classes from Requirements`

Session 1 Outcomes

By the end of today, learners will be able to:

- Set up an advanced-course capstone workspace and Git baseline
- Use Git as a checkpoint system with meaningful commit messages
- Translate a requirements list into domain models and service boundaries
- Protect object state with validated properties and specific custom exceptions
- Choose composition over inheritance when collaborator behavior may change

Scope Guardrails for Today

In Scope

- Project scaffold and repeatable workflow habits
- Responsibility-driven design and boundary thinking
- Properties, invariants, and domain-specific exceptions
- Lightweight polymorphism and composition patterns

Not Yet Covered

- Database or file-based persistence (Day 3+)
- GUI or HTTP interface layers
- Production authentication or security systems
- Abstract base classes, protocols, or deep inheritance hierarchies

Hour 1: Advanced Kickoff + Baseline Diagnostic

Learning Outcomes

- Explain the difference between file save and lab session save
- Create the standard capstone folder layout with Git initialized
- Write and run a deterministic diagnostic script covering class, dict, file, and exception
- Make a first meaningful Git commit with a clear message

Source Alignment

- `Advanced/lessons/lecture/Day1_Hour1_Advanced.md` → `## Learning Outcomes`

Why Day 1 Starts Here

Advanced Python is about **intentional structure**, not just syntax.

- The course builds a capstone tracker incrementally across all sessions
- Working in small steps and committing frequently reduces recovery pain
- A clean project layout now makes future features easier to add
- Today's goal is a repeatable workflow before the architecture grows

> "Advanced means intentional. We want code that communicates clearly."
>
> — `Advanced/lessons/lecture/Day1_Hour1_Advanced.md` → `## Opening Script`

Two Kinds of Saving

Understanding this distinction prevents lab-session loss:

Action	What it saves
`Ctrl+S` or File → Save	The current file on disk
Save and Suspend	The entire lab session state

- Saving a file is not the same as suspending the lab
- If stepping away for more than ~15 minutes, use **Save and Suspend**
- Locate the Save and Suspend control before writing any code

Source Alignment

- `Advanced/lessons/lecture/Day1_Hour1_Advanced.md` → `## LogicalLabs/CHOICE Launch and Persistence Briefing`

Standard Capstone Scaffold

```
advanced_tracker/
├── src/          ← application code
├── tests/        ← test files
├── data/         ← small local data files
└── reports/      ← generated outputs
```

- Use `.gitkeep` placeholder files in empty folders so Git tracks them
- Keep course work inside one consistent repo/workspace
- Build the folder habit now — the project will grow into these directories

Git Workflow for This Course

Six words to know from day one:

- **clone** — copy a remote repo to the local workspace
- **pull** — bring in remote changes before starting work
- **add** — stage changes for the next commit
- **commit** — record a local checkpoint in history
- **push** — send local commits to a remote (GitHub etc.)
- **status** — inspect what Git currently sees (`git status` is your friend)

```
git init
git status
git add diagnostic.py src/.gitkeep tests/.gitkeep reports/.gitkeep
git commit -m "chore: initial setup"
git status
```

Demo: Diagnostic Script

```python
from pathlib import Path


class LearnerProfile:
    def __init__(self, name: str, goal: str) -> None:
        self.name = name
        self.goal = goal

    def summary(self) -> str:
        return f"{self.name} is practicing: {self.goal}"


def main() -> None:
    profile = LearnerProfile("Advanced Learner", "readable, testable Python")
    status = {"python": "ready", "git": "ready for first commit"}
    report_path = Path("data") / "diagnostic_report.txt"

    try:
        report_path.write_text(profile.summary() + "\n", encoding="utf-8")
        print(report_path.read_text(encoding="utf-8"))
        print("Diagnostic complete.")
    except OSError as error:
        print(f"File operation failed: {error}")


if __name__ == "__main__":
    main()
```

Lab: Environment + Baseline Diagnostic

Time — 25–35 minutes

Tasks

1. Launch the assigned LogicalLabs/CHOICE lab; locate Save and Suspend
2. Create a `course_preflight.txt` with your name, date, and one learning goal
3. Create `advanced_tracker/` with subfolders `src/`, `tests/`, `data/`, `reports/`
4. Initialize Git and add `.gitkeep` files in the empty subdirectories
5. Write `diagnostic.py` that exercises a class, a dictionary, file I/O, and a `try/except`
6. Run the script cleanly, then commit with `git commit -m "chore: initial setup"`

Completion Criteria — Hour 1

✓ `python --version` succeeds in the lab terminal
✓ Project folders match the standard scaffold
✓ `git init` completed and first commit recorded
✓ Diagnostic script runs without a traceback
✓ Learner can explain the difference between **commit** and **push**

Common Pitfalls — Hour 1

⚠️ Confusing `Ctrl+S` file save with lab session Save and Suspend
⚠️ Accumulating three hours of changes before making a single commit
⚠️ Skipping the diagnostic script and discovering environment issues in Hour 2
⚠️ Using `git add .` from the wrong directory and staging unexpected files

Quick Check

Question: What does `git commit` do that `git add` does not?

Hour 2: Designing Classes from Requirements

Learning Outcomes

- Translate a short requirements list into candidate classes
- Describe any class in terms of what it **knows** and what it **does**
- Keep `input()` and `print()` out of domain model classes
- Implement a model class and a coordinating service function

Source Alignment

- `Advanced/lessons/lecture/Day1_Hour2_Advanced.md` → `## Learning Outcomes`

Design Vocabulary

Four terms used all course:

- **Requirement** — something the program must do, from the outside view
- **Responsibility** — a job owned by a piece of code; ask *knows* and *does*
- **Boundary** — a line between responsibilities that lets one part change without breaking others
- **Service function** — a small coordinating function that uses model objects to complete one use case

> "A good class definition is not only a list of what belongs. It is also a boundary against what does not belong."
>
> — `Advanced/lessons/lecture/Day1_Hour2_Advanced.md` → `## Concept Briefing: From Requirements to Classes`

From Requirements to Classes — Five Steps

```
Step 1 — Underline important nouns and verbs
Step 2 — Group related data and behavior
Step 3 — Name candidate classes
Step 4 — For each class, list what it knows and does
Step 5 — Decide what does NOT belong in the class
```

- Nouns are clues, not automatic classes
- Avoid the "junk drawer" — one giant manager that does everything
- A small testable class beats a large untestable god-object every time

Source Alignment

- `Advanced/lessons/lecture/Day1_Hour2_Advanced.md` → `## Concept Briefing: From Requirements to Classes`

Sketching the Tracker Domain

```
Task
  knows — title, priority, completed status
  does  — mark_complete(), is_pending(), change_priority()
  not   — ask user for input, print itself to screen

Project
  knows — name, list of Task objects
  does  — add_task(), pending_tasks(), completed_count()
  not   — display menus, read from input()

Service function
  complete_task_by_title(project, title)
  — searches the project, calls task.mark_complete()
  — returns True/False; does not print
```

Demo: Task + Project Classes

```python
from __future__ import annotations


class Task:
    allowed_priorities = {"low", "medium", "high"}

    def __init__(self, title: str, priority: str = "medium") -> None:
        cleaned = title.strip()
        if not cleaned:
            raise ValueError("Task title cannot be empty.")
        if priority.strip().lower() not in self.allowed_priorities:
            raise ValueError(f"Priority must be one of: {sorted(self.allowed_priorities)}")
        self.title = cleaned
        self.priority = priority.strip().lower()
        self.is_complete = False

    def mark_complete(self) -> None:
        self.is_complete = True

    def is_pending(self) -> bool:
        return not self.is_complete

    def __str__(self) -> str:
        status = "done" if self.is_complete else "pending"
        return f"{self.title} [{self.priority}, {status}]"
```

Demo: Project + Service Function

```python
class Project:
    def __init__(self, name: str) -> None:
        self.name = name.strip()
        self.tasks: list[Task] = []

    def add_task(self, task: Task) -> None:
        self.tasks.append(task)

    def pending_tasks(self) -> list[Task]:
        return [t for t in self.tasks if t.is_pending()]

    def completed_count(self) -> int:
        return sum(1 for t in self.tasks if t.is_complete)


def complete_task_by_title(project: Project, title: str) -> bool:
    for task in project.pending_tasks():
        if task.title.lower() == title.strip().lower():
            task.mark_complete()
            return True
    return False
```

Demo: Running the Tracker

```python
def main() -> None:
    project = Project("Day 1 Tracker")
    project.add_task(Task("Sketch tracker classes", "high"))
    project.add_task(Task("Implement one model", "medium"))
    project.add_task(Task("Separate UI from logic", "high"))

    completed = complete_task_by_title(project, "Implement one model")
    print(f"Completed requested task: {completed}")
    print(f"Pending: {len(project.pending_tasks())} tasks remaining")
    for task in project.pending_tasks():
        print(f"  - {task}")
```

Expected output:

```
Completed requested task: True
Pending: 2 tasks remaining
  - Sketch tracker classes [high, pending]
  - Separate UI from logic [high, pending]
```

Lab: Domain Modeling

Time — 25–35 minutes

Tasks

1. Choose one tracker domain — tasks, contacts, inventory, expenses, or notes
2. Write 5–8 plain-language requirements
3. Design 3–5 candidate classes (what each *knows* and *does*)
4. Implement at least one domain class and one coordinating class or service function
5. Write a `main()` demo block with no `input()` or `print()` inside the model

Completion Criteria + Common Pitfalls — Hour 2

✓ Requirements map to classes with clear responsibilities
✓ No `input()` or `print()` inside domain class methods
✓ At least one working model class and one service function
✓ `main()` creates objects and shows a summary externally

⚠️ Putting UI prompts or `print()` inside model methods
⚠️ Building one giant class that coordinates, stores, and displays everything
⚠️ Jumping to inheritance before understanding what each class owns
⚠️ Skipping Step 5 — not deciding what does NOT belong in a class

Quick Check

Question: If a `Contact` class calls `input("Email: ")` in its `__init__`, what does that make harder?

Hour 3: Properties, Invariants, and Custom Exceptions

Learning Outcomes

- Explain a property as controlled access to an attribute
- Use a backing attribute to avoid recursive property calls
- Define custom domain exceptions for validation failures and missing items
- Update a service function to raise domain exceptions instead of returning `None`

Source Alignment

- `Advanced/lessons/lecture/Day1_Hour3_Advanced.md` → `## Learning Outcomes`

Core Concept — Invariants and Properties

An **invariant** is a rule that must always be true for an object to be valid.

Examples for a task tracker:

- Title must not be blank or whitespace-only
- Priority must be an integer between 1 and 5
- Completed status must never be set to an arbitrary type

A **property** creates one safe doorway into a piece of state:

- Every assignment — including during `__init__` — passes through the setter
- The setter uses a **backing attribute** (`_priority`) to store the real value
- Writing `self.priority = value` inside the setter causes infinite recursion — always use `self._priority = value`

Demo: Custom Domain Exceptions

```python
class ValidationError(Exception):
    """Raised when tracker data violates a domain rule."""


class NotFoundError(Exception):
    """Raised when a requested tracker item does not exist."""
```

Returning `None` or `False` for domain failures:

- The caller might forget to check the return value
- The error meaning is ambiguous
- The real failure can travel silently and surface far from its cause

Raising a specific exception gives the caller a **clear, catchable signal**.

Source Alignment

Demo: Validated Task Model

```python
class Task:
    def __init__(self, title: str, priority: int = 3) -> None:
        self.title = title       # routes through the setter
        self.priority = priority  # routes through the setter
        self.completed = False

    @property
    def title(self) -> str:
        return self._title

    @title.setter
    def title(self, value: str) -> None:
        if not isinstance(value, str) or not value.strip():
            raise ValidationError("Task title cannot be empty.")
        self._title = value.strip()

    @property
    def priority(self) -> int:
        return self._priority

    @priority.setter
    def priority(self, value: int) -> None:
        if type(value) is not int or not (1 <= value <= 5):
            raise ValidationError("Task priority must be an integer from 1 to 5.")
        self._priority = value
```

Demo: Service with NotFoundError

```python
def complete_task_by_title(tasks: list[Task], title: str) -> Task:
    """Mark the first matching task complete; raise NotFoundError if missing."""
    normalized = title.strip()
    for task in tasks:
        if task.title.casefold() == normalized.casefold():
            task.mark_complete()
            return task
    raise NotFoundError(f"No task found with title: {title!r}")
```

The service raises `NotFoundError` — it does **not** print, return `False`, or return `None`. The caller decides how to communicate the failure.

Demo: Caller-Level Error Handling

```
tasks = [Task("Write outline", 2), Task("Review PR", 4)]

try:
    done = complete_task_by_title(tasks, "review pr")
    print(f"Completed: {done.title}")
except NotFoundError as error:
    print(f"Could not complete task: {error}")

try:
    Task("   ", priority=2)          # blank title
except ValidationError as error:
    print(f"Could not create task: {error}")

try:
    complete_task_by_title(tasks, "Submit invoice")
except NotFoundError as error:
    print(f"Could not complete task: {error}")
```

- Catch **specific** exceptions only — not `except Exception`

Lab: Validation + Exceptions

Time — 25–35 minutes

Tasks

1. Add at least one `@property` with a validated setter to a model class
2. Use a backing attribute (`_title`, `_amount`, etc.) in every setter
3. Define `ValidationError` and `NotFoundError` as custom exception classes
4. Raise `ValidationError` in the setter when the value breaks the invariant
5. Update one service function to raise `NotFoundError` for a missing item
6. Write a `main()` block that catches specific custom exceptions and prints user-friendly messages

Completion Criteria + Common Pitfalls — Hour 3

✓ A real domain invariant is enforced in a `@property` setter
✓ The setter stores the value in a backing attribute, not `self.property`
✓ Both `ValidationError` and `NotFoundError` are defined and raised intentionally
✓ Friendly messages appear only at the caller boundary, not inside the model

⚠️ Writing `self.priority = value` inside `priority.setter` — causes recursion
⚠️ Returning `False` or `None` instead of raising a domain exception
⚠️ Using `except Exception` broadly, which hides unrelated programming bugs
⚠️ Validating only in the UI layer and skipping model-level protection

Quick Check

Question: Why is `ValidationError` raised in the setter better than returning `False` from a validate method?

Hour 4: Composition vs Inheritance + Polymorphism

Learning Outcomes

- Distinguish **is-a** (inheritance) from **has-a** (composition)
- Explain duck typing — Python cares about available behavior, not ancestry
- Identify `if/elif` chains that are candidates for polymorphic refactoring
- Refactor a branching exporter into composed formatter objects

Source Alignment

- `Advanced/lessons/lecture/Day1_Hour4_Advanced.md` → `## Learning Outcomes`

Inheritance vs Composition

```
Inheritance — use when one type really IS a specialized version of another
  Test: "A SavingsAccount IS a BankAccount" ✓ reasonable

Composition — use when one object HAS or USES another object
  Test: "A ReportRunner HAS a formatter" ✓ correct
        "A ReportRunner IS a formatter"  ✗ wrong relationship
```

Four vocabulary items:

- **Inheritance** — subclass receives structure and behavior from a parent
- **Composition** — one object stores or receives another object as a collaborator
- **Polymorphism** — different objects respond to the same method name
- **Duck typing** — if an object provides the method the caller needs, it qualifies

Source Alignment

Design Smell — The Branchy Exporter

```python
from typing import Any


def export_report(report: dict[str, Any], format_type: str) -> object:
    if format_type == "text":
        return f"Report: {report['title']}\nCompleted: {report['completed']}"
    elif format_type == "dict":
        return {"title": report["title"], "completed": report["completed"]}
    else:
        raise ValueError(f"Unknown format: {format_type}")
```

The smell:

- Every new format requires reopening and editing this function
- A misspelled string silently reaches the `else` branch
- Changing one branch risks breaking another

Demo: Polymorphic Formatter Classes

```python
from typing import Any

Report = dict[str, Any]


class TextReportFormatter:
    def format(self, report: Report) -> str:
        return (
            f"Report: {report['title']}\n"
            f"Completed: {report['completed']}\n"
            f"Pending: {report['pending']}"
        )


class DictReportFormatter:
    def format(self, report: Report) -> dict[str, Any]:
        total = report["completed"] + report["pending"]
        return {
            "title": report["title"],
            "completed": report["completed"],
            "pending": report["pending"],
            "total": total,
        }
```

Demo: ReportRunner Using Composition

```python
class ReportRunner:
    def __init__(self, formatter: Any) -> None:
        self.formatter = formatter   # has-a relationship

    def build_output(self, report: Report) -> object:
        return self.formatter.format(report)   # polymorphic call


def main() -> None:
    report = {"title": "Day 1 Tracker", "completed": 7, "pending": 3}

    text_runner = ReportRunner(TextReportFormatter())
    dict_runner = ReportRunner(DictReportFormatter())

    print(text_runner.build_output(report))
    print()
    print(dict_runner.build_output(report))
```

`ReportRunner` does not ask *which kind* of formatter it holds.

Adding a Third Formatter — No Changes to ReportRunner

```
class SummaryReportFormatter:
    def format(self, report: Report) -> str:
        total = report["completed"] + report["pending"]
        return f"{report['title']}: {report['completed']} of {total} complete"


summary_runner = ReportRunner(SummaryReportFormatter())
print(summary_runner.build_output(report))
# Day 1 Tracker: 7 of 10 complete
```

This is the payoff: a new behavior is added by creating a new class.

`ReportRunner.build_output()` is **never modified**.

Source Alignment

- `Advanced/lessons/lecture/Day1_Hour4_Advanced.md` → `### Step 4: Optional quick extension`

Lab: Refactor a Branching Feature

Time — 25–35 minutes

Tasks

1. Start with a working `if/elif` function (export, notification, or payment)
2. Refactor each branch into its own class with one shared method name (`format`, `send`, or `process`)
3. Create a runner/service class that **has-a** collaborator and calls the shared method
4. Swap collaborators in the demo block without changing the runner's internals
5. Optional — add a third implementation to confirm zero changes to the runner

Completion Criteria + Common Pitfalls — Hour 4

✓ A large `if/elif` chain is removed from the caller
✓ Two or more classes share the same method name
✓ The runner/service uses composition — it stores a collaborator, not inherits from one
✓ A third implementation can be added without touching the runner

⚠️ Moving the `if/elif` chain into a new class — that relocates branching, not removes it

⚠️ Inheriting from a formatter when the real relationship is has-a

⚠️ Assuming polymorphism requires a shared base class — duck typing handles it in Python

⚠️ Hard-coding the collaborator choice inside the runner constructor

Quick Check

Question: What is the practical benefit of giving the runner *any* object with a compatible `format()` method?

Session 1 Wrap-Up

What We Built Today

Hour	Deliverable
Hour 1	Stable workspace and first Git commit
Hour 2	Domain model with clear responsibilities and boundaries
Hour 3	Validated properties and domain-specific exceptions
Hour 4	Composition-based design with polymorphic collaborators

Key Day 1 Rule — Professional Python starts with trustworthy structure.

Day 1 Checklist + Next Session Preview

Before Session 2

- Re-run the diagnostic script to confirm the environment is still ready
- Add a validated `@property` to at least one model attribute
- Define `ValidationError` and `NotFoundError` and raise them intentionally
- Refactor one `if/elif` workflow into composed collaborators
- Commit every working checkpoint with a meaningful message

Session 2 Preview (Hours 5–8)

- Factory pattern for centralized, validated object creation
- Strategy pattern for runtime-swappable behavior
- `__repr__`, `__str__`, equality, and light type hints
- Checkpoint 1 — stable headless core: models, service CRUD, exceptions

Thank You!

Save your files.

Commit the latest working state.

Be ready to extend the core tomorrow.